GENE-QUERY JEPA

Why the gene MSE does not fall

A note on the two source-blind runs, 8–9 October 2026

Two trainings are in progress. Both freeze the encoder after its first three layers, train a three-layer predictor, and draw queries without looking at the target cell: 96 genes from the source list and 32 from outside it. The objective is

$$L = L_var + 0.25\, L_abs + 0.1\, L_NCE + 0.1\, L_mask$$

averaged over the target times. The cell question has weight zero. One run starts from a resting cell and predicts 90 minutes, 6 hours and 10 hours. The other starts from a 90-minute cell and predicts 6 hours and 10 hours. The learning rate is 10^{-4}, and both are scheduled for 100 epochs.

On the morning of 9 October the resting-source run had finished validation at epoch 42, and the 90-minute run at epoch 61. In both, the validation gene mean squared error has been still since epoch 0.

Run	Epoch 0	Latest	Cosine	Gap vs. copy
Resting source	0.290	0.284 (ep. 42)	0.846	0.059
90-minute source	0.289	0.289 (ep. 61)	0.843	0.057

Cosine and the gap against copying the source row are validation figures at the latest epoch above. The gene loss has not followed the auxiliary mask task. On the 90-minute run the training mask cross-entropy is 1.12 at epoch 61, while validation has turned back up to 4.39.

The floor has three parts

The validation gene loss is the mean, over queries whose gene is present at the target time and which are not mask questions, of the mean squared coordinate error between the raw prediction and the target row. In 768 dimensions that is the squared length of the error, divided by 768. The value near 0.28 is three contributions stacked on top of each other.

- **About 0.19 is label noise, and it cannot be trained away.**
  A source cell is paired with a target cell of the same type drawn at random. Nothing shown to the predictor identifies that particular target cell, so the best available answer is the mean row of the gene at that cell type and time. On the 1 October controls, two cells of the same type had a cosine of about 0.81 between the target-encoder rows of the same gene. Those rows leave a layer normalisation, so their squared length is about 768. The expected squared distance from either row to their mean is then $768 \times (1 - 0.81)$, and dividing by 768 leaves 0.19. That is the error of predicting one cell's row from another cell of the same type. No change of depth, learning rate, or number of epochs moves it, because the missing information is not in the label.

- **The conditional mean, written down as a table, scores about 0.22.**
  On the same 1 October controls a table of one mean vector per gene, cell type and time reached a mean squared error of 0.22, and it beat the trained predictor, which was then near 0.29. The step from 0.19 up to 0.22 is the cost of estimating the table, and of conditioning only on type and time. These two figures were not recomputed on the runs that are training now.
- **The predictor sits about 0.06 above that table.**
  Today's runs are at 0.284 and 0.289. The network has already collapsed onto a type-level answer. On the 1 October controls, its predictions for the same gene, type and time in two different cells had cosine about 0.996. It is emitting the mean. It is not landing on the mean exactly, and two pieces of the recipe hold it off.

The contrastive term pulls against the mean. Its negatives are the other present queries in the batch, including the same gene in other cells, and those target rows are only about 0.81 apart. The squared loss asks the prediction to sit at the average. The contrastive loss, with weight 0.1 and temperature 0.1, asks those predictions to come apart. There is no cell-specific direction for them to come apart along. Validation InfoNCE has stayed near 6.25 for the whole of both runs, which is what a constant repulsion looks like when the positives and the negatives are nearly the same. The term holds the squared error above the table.

The last layer normalisation fixes the length of every query. An average of several rows is shorter than a typical row, by about the factor $\sqrt{0.81}$. A layer norm on the decoder output does not let a query take that shorter length, so the prediction cannot sit on the mean even in scale. The effect is small, on the order of 0.01, and it falls on the genes whose rows vary the most.

What would actually move it

Further epochs will not. The two runs already agree with each other, and they agree with the shorter run from 1 October. They also agree across source time. A 90-minute cell is closer to 6 hours and 10 hours than a resting cell is, and the gene loss at those times is nevertheless the same. The source cell is being used as a cue for type, and little more.

0 **Measure the ceiling before touching the trainer.**
On the validation split, and inside each cell type and time, match source cells to target cells by the frozen encoder's CLS vector. A nearest neighbour is enough. Then, on a few hundred genes, compare the cosine of matched rows with the cosine of random rows. The random figure is the 0.81 already in hand. If matched pairs reach 0.88 or better, transport matching is worth a training run. If they stay near 0.82, it is not, and the retraining below should be skipped. This is a short job on one GPU, and it should come before any new training.

1 **Replace random pairs by optimal transport.**
Match inside each cell type and time so that every target cell is used once. This is the only change that adds information. It lowers the label noise itself, from $1 - 0.81$ to $1 - c$, where c is the cosine of the matched pairs. A matched cosine of 0.90 would take that floor from 0.19 to 0.10. The change belongs in the code that builds the pairs, not in the network.

2 **Set the InfoNCE weight to zero.**
Leave the squared error as the only pressure on genes that are present. The expected movement is small, about 0.01 to 0.04, down into the middle of the 0.25s. It removes the term that is currently arguing with the mean.

3 **If a lower curve is wanted for the report, average four targets.**
Replace each single target row by the mean of four random cells of the same type. The noise term falls by about four, from 0.19 to roughly 0.05, and the validation error should land near 0.10 to 0.13. This teaches the model nothing new. It scores the model on the average it has already learned. Knockout means and gene-program scores are themselves averages over a cell type, so nothing those analyses use is discarded. The change is legitimate, and it has to be described as a change of the label.

4 **A small correction to the norm.**
Replace the final layer norm on each query by a predicted scale, so a query can take the shorter length of a mean. This is worth about 0.01, mostly on genes whose rows vary.

What the next run should not be

Do not add epochs, sweep the learning rate, or widen the predictor in the hope of a lower gene loss. Do not unfreeze the encoder for that purpose either. If the encoder is allowed to move, it can reduce the loss by making the target rows more alike, which removes the within-type variation rather than predicting it. The number would fall because the target had become poorer.

The recommendation

Run the matching measurement first. If matched pairs clear a cosine of 0.88, retrain with transport pairs and with the contrastive weight at zero. That is the combination expected to move the loss for a real reason, from about 0.28 into a band around 0.15, give or take about 0.03. If the only aim is the curve in the report, the average of four targets is cheaper and moves the printed number further, and the figure should say that the label was changed.

The cosine of 0.81, the mean-table error of 0.22, and the within-type prediction cosine of 0.996 are from the 1 October controls. The epoch-0 and latest losses, the copy gaps, and the mask cross-entropy are from the two runs training on 8–9 October.